

Informe de equipo Marrón (27 y 03)

Roles designados

Los roles son los descritos por el enunciado del trabajo practico y fueron designados de la siguiente manera

- Ingeniero de comunicaciones en C: Luciano Belardo
- Gestor de recursos y estado: Adrian David Albahaca Chavez
- Planificador y lógica anti-deadlock: Francisco Ares
- Integración, pruebas e interoperabilidad: Genaro Lescano

El agente-C

En el trabajo realizado, el agente C, implementado en server.c, es el corazón de las comunicaciones del sistema. Actúa como servidor (o sea, escucha peticiones) y como cliente (o sea, inicia conexiones). Además, El agente C también realiza una gestión de recursos y estado sobre sí mismo, simulando un nodo HPC (High Performance Computing) dentro del clúster. Su propósito es:

- **Administrar los recursos locales:** CPUs, memoria y GPUs disponibles en el nodo.
- **Mantener un inventario actualizado:** Registrar qué recursos están disponibles, cuáles están reservados y por qué job.
- **Simular un entorno real de HPC:** Donde cada nodo tiene recursos limitados que deben ser gestionados y compartidos entre múltiples jobs.
- **Proporcionar información al planificador Erlang:** Para que tome decisiones sobre qué jobs ejecutar y en qué nodos.

en esta sección informaremos sobre las técnicas y arquitecturas empleadas sobre el agente C, en el aspecto de la comunicación con epoll y sockets no-bloqueantes como la propia gestión de recurso y estado que realiza sobre si mismo el agente C.

Arquitectura de comunicaciones del agente C: Implementación con epoll y sockets No-bloqueantes

Como dijimos, el agente C es el corazón de las comunicaciones del sistema, actúa como servidor y como cliente, además integra un módulo de descubrimiento basado en UDP broadcast que permite la detección de nodos en la red. Por lo que maneja tres tipos de comunicaciones

- TCP público: con otros agentes C (remoto)
- TCP local: con el planificador erlang (localhost)
- UDP broadcast: para descubrimiento de nodos

Desarrollamos

➔ **Estructura connection_t y sus campos:** encapsula el estado de cada conexión activa.

connection_t

```

└─ fd: int                // file descriptor del socket
└─ type: connection_type_t //tipo de conexión (remota/local/escucha)
└─ read_buf[BUFF_SIZE]    // buffer para datos recibidos
└─ read_pos: size_t        //posición actual en el buffer de lectura
└─ write_mutex: pthread_mutex_t // mutex para acceso seguro al buffer de escritura
└─ write_buf[BUFF_SIZE]    // buffer para datos a enviar
└─ write_pos: size_t        //posición actual en el buffer de escritura
└─ write_len: size_t        //cantidad total de datos en el buffer
(Diagrama de los campos de la estructura connection_t)

```

Incluye el file descriptor (fd), el tipo de conexión (connection_type_t), y buffers para lectura y escritura con sus respectivos punteros de posición. Su funcionamiento detallado se explica en las secciones **Manejo de Lectura y Escritura** y **Estrategia de Envío de Mensajes**. Sin estos campos los mensajes fragmentados se perderían y se necesitarían lecturas bloqueantes. El uso de un mutex (write: mutex) protege el buffer de escritura ante accesos simultáneos desde diferentes hilos.

- ➔ **Inicialización y configuración de sockets:** la inicialización del servidor se realiza mediante `init_server_socket()`, que crea tres sockets de escucha: dos TCP (público y local) y uno UDP para descubrimiento. Cada socket se configura con la opción `SO_REUSEADDR`, (se trata de una opción de socket que permite reutilizar direcciones locales que están en estado `TIME_WAIT` donde normalmente cuando un socket se cierra el sistema operativo mantiene el puerto en estado `TIME_WAIT` por un tiempo determinado, con `SO_REUSEADDR` el kernel permite que otro socket se bindee a la misma dirección/puerto aun que haya un socket en `TIME_WAIT`) para permitir reinicios rápidos (pues en caso contrario deberíamos esperar a que las direcciones salgan del estado `TIME_WAIT`) y se establece en modo no-bloqueante mediante `fcntl`, el cual permite que operaciones como `read()`, `write()`, `accept()` y `connect()` retornen valores de manera inmediata en vez de bloquearse y esperar a que sean efectivamente empleadas. Posteriormente, los sockets se registran en la instancia de `epoll` con eventos `EPOLLIN`, utilizando `EPOLLEXCLUSIVE` (flag para distribuir eficientemente las conexiones entrantes entre múltiples hilos y de esta manera el kernel despierta un solo hilo para cada nueva conexión por lo que se reduce drásticamente el overhead de CPU) para los sockets de escucha, garantizando una distribución eficiente de las conexiones entre los hilos de trabajo.
- ➔ **Mecanismo de epoll y Thread pool:** El servidor utiliza un pool de `NUM_THREADS` hilos, cada uno ejecutando `worker_thread_loop()`, su trabajo es monitorear eventos y procesarlos de manera continua. Este bucle principal utiliza `epoll_wait()` para monitorear eventos en los file descriptor registrados. Se emplea la bandera `EPOLLONESHOT`, flag que deshabilita el descriptor después de que se genera un evento, hasta que sea reactivado explícitamente, para garantizar que cada evento sea procesado por un único hilo, evitando condiciones de carrera, pues en caso contrario múltiples hilos podrían procesar el mismo evento simultáneamente provocando dobles procesamiento generando datos corruptos o respuestas duplicadas, race condition generando lectura/escritura inconsistente y perdida de datos pues un hilo lee y el otro también provocando que algunos bytes pueden perderse entre otros. Tras procesar un evento, el descriptor debe ser rearmado explícitamente (pues `EPOLLONESHOT` lo

desactiva) mediante `epoll_ctl()` con la máscara de eventos actualizada , lo que proporciona un control fino sobre el flujo de procesamiento

`epoll_wait()` → Evento recibido

- └─ `EPOLLIN` → `handle_tcp_read()` / `handle_udp_read()` / `handle_accept()`
- └─ `EPOLLOUT` → `handle_tcp_write()` / `handle_outgoing_connection_event()`
- └─ `EOLLERR` → `process_disconnect()`
- └─ `EPOLLHUP` → `process_disconnect()`

(diagrama de flujo del mecanismo)

- **Manejo de Conexiones Entrantes (Servidor):** Las conexiones entrantes son aceptadas mediante `handle_accept()` que itera sobre `accept()` hasta que retorna `EAGAIN` (valor de retorno de `accept()` en modo no-bloqueante cuando no encuentra más conexiones), asegurando que se vacíe completamente la cola de conexiones pendientes. Para cada nueva conexión, se crea una estructura `connection_t` y se determina su tipo basado en el socket de escucha original:
`CONN_TCP_CLIENTE_REMOTE` para conexiones de otros agentes C, o
`CONN_TCP_CLIENT_LOCAL` para conexiones del planificador Erlang. El nuevo descriptor se registra en `epoll` con `EPOLLIN` | `EPOLLONESHOT`, listo para recibir datos.
- **Manejo de Conexiones Salientes (Cliente):** Para establecer conexiones salientes, el agente utiliza `connect_remote_node()`, que crea un socket no-bloqueante e inicia un `connect()` asíncrono (conexión que no espera a que se establezca, solo se inicia el proceso y continua con otras tareas). dado que `connect()` retorna inmediatamente con `EINPROGRESS` (indica que la operación se inició correctamente y está en progreso), el descriptor se registra en `epoll` con `EPOLLOUT`. Cuando el handshake TCP se completa (cuando la conexión con otra computadora se ha establecido), `handle_outgoing_connection_event()` verifica el éxito de la conexión mediante `getsockopt(SO_ERROR)` ya que estamos utilizando un `connect()` asíncrono. `Getsockopt(SO_ERROR)` verifica que la conexión iniciada ah fallado o funcionado y reconfigura el descriptor para operaciones de lectura regulares (`EPOLLIN`), notificando al gestor de recursos a través de `process_connection_ready()`.
- **Manejo de Lectura y Escritura:** La lectura de datos TCP se implementa en `handle_tcp_read()`, que almacena los bytes recibidos en el buffer interno `connection_t -> read_buf` a partir de la posición `connection_t -> read_pos`. Este enfoque es necesario porque TCP es un flujo de bytes sin delimitación de mensajes; un único `read()` puede contener mensajes parciales o múltiples mensajes concatenados. La función extrae mensajes completos delimitados por `\n` mediante `strchr()`, procesándolos individualmente a través de `process_message()`. Los mensajes parciales se preservarán al inicio del buffer para ser completados en futuras lecturas. Similarmente, `handler_tcp_write()` gestiona la escritura, manteniendo un buffer de salida (`connection_t -> write_buf`) y actualizando la posición de escritura (`connection_t -> write_pos`) hasta que todos los datos sean transmitidos (Explicado en **Estrategia de Envío de Mensajes**). En ambos casos, luego de procesar la operación, se rearma el descriptor en `epoll` con la máscara de eventos adecuada (`EPOLLIN` o `EPOLLIN` | `EPOLLOUT` según haya datos pendientes).

Read: "RESERVE 1001 cpu 2\nRESER" → extrae primer mensaje, guarda "RESER"

Read: "VE 1002 mem 4\n" → concatena "RESERVE 1002 mem 4\n" → extrae mensaje completo

(caso de ejemplo)

- ➔ **Manejo de Desconexiones y Limpieza:** El sistema maneja robustamente las desconexiones inesperadas. Cuando `read()` retorna 0, se interpreta como un cierre ordenado del socket; mientras que eventos `EPOLERR` o `EPOLLHUP` indican fallos de conexión. En ambos casos, se invoca `process_disconnect()` para notificar al gestor de recursos y liberar los recursos asociados al job. Finalmente, el socket se cierra y la estructura `connection_t` se destruye, incluyendo el mutex de escritura.
- ➔ **Mecanismo de Descubrimiento UDP:** el descubrimiento de nodos se implementa mediante UDP broadcast (mecanismo que permite enviar un datagrama a todos los dispositivos en la misma red local). El socket UDP se registra en `epoll` y `handle_udp_read()` procesa todos los datagramas pendientes en el buffer del sistema operativo mediante `recvfrom()` hasta que retorna `EAGAIN`. Cada mensaje se parsea y se pasa a `process_announce()`, que actualiza la tabla de nodos conocidos. Periódicamente, el agente emite anuncios de presencia mediante `send_udp_broadcast()` el cual crea un socket UDP temporal, habilita broadcast, configura el destino, envía el mensaje a todos los nodos y cierra el socket, permitiendo que otros nodos detecten su disponibilidad.
- ➔ **Estrategia de envío de Mensajes:** el envío de mensajes se canaliza a través de `enqueue_write()`, que copia el mensaje en el buffer de escritura de la conexión (`connection_t`), en `connection_t -> write_buf`. El buffer mantiene un puntero de escritura, `connection_t -> write_pos`, que indica hasta donde se ha transmitido, y la longitud total, en `connection_t -> write_len`, que indica cuanto queda por enviar. Esto permite manejar situaciones donde el socket no puede aceptar todos los datos de una vez (por ejemplo, cuando el buffer del sistema operativo está lleno), permitiendo reanudar la transmisión más tarde sin pérdida de datos. Esta función es thread-safe gracias al uso de `write_mutex` pues evita que dos hilos escriban al mismo tiempo y corrompan los datos. Tras añadir el mensaje, se modifica la máscara de eventos en `epoll` para incluir `EPOLLOUT`, lo que despertara al hilo correspondiente para ejecutar `handle_tcp_write()` y transmitir los datos.

Gestor de recursos y Estado del agente C

El agente C no solo maneja las comunicaciones, sino que también realiza una gestión completa de recursos locales y mantiene el estado del sistema, simulando un nodo HPC real dentro del clúster distribuido. Esta capa es fundamental para la asignación, seguimiento y liberación de recursos, así como para mantener la consistencia del sistema ante fallos.

- **Recursos Locales:**

```
typedef struct {  
    int available_amount;    // Cantidad disponible actual  
    int total_amount;        // Capacidad total del recurso  
    ColaPendingRequest cola; // Cola FIFO de solicitudes pendientes  
    pthread_mutex_t mutex;   // Protección para acceso concurrente  
} Recurso;
```

El sistema maneja tres tipos de recursos (`RESOURCE_CPU`, `RESOURCE_MEM`, `RESOURCE_GPU`), cada uno como una instancia independiente de esta estructura. Los recursos se inicializan con la capacidad total definida al iniciar el agente y mantienen un seguimiento en tiempo real de la cantidad disponible.

- **Tabla de Jobs Activos**

```
typedef struct {
    Job* tabla_jobs[TAM_TABLA_JOBS]; // Hash table de jobs activos
    pthread_mutex_t mutex;           // Protección para acceso concurrente
} TablaJobs;
```

Esta tabla hash mantiene un registro de todos los jobs activos en el sistema, cada job_id está asociado con la conexión del cliente que solicitó el job (connection_t *conn) y la lista de recursos asignado al job (Allocation *allocations).

- **Tabla de nodos conocidos**

```
typedef struct {
    char ip[16];           // Dirección IP del nodo
    int puerto;            // Puerto TCP del agente
    int cpu_disp, mem_disp, gpu_disp; // Recursos disponibles
    time_t last_seen;      // Último anuncio recibido
    struct NodeEntry *next; // Siguiendo entrada en la lista
} NodeEntry;
```

Almacena los nodos descubiertos mediante UDP broadcast, manteniendo información actualizada de sus recursos disponibles y el momento del último anuncio. Los nodos que no envían anuncio en un periodo de 15 segundos son automáticamente purgados.

- **Mecanismo de Reserva de Recursos:** cuando el agente recibe una solicitud RESERVE (desde otro nodo) o un JOB_REQUEST (desde Erlang), el proceso de reserva sigue el siguiente flujo:

```
static result_t reserve_resource(resource_t tipo, int amount, int job_id, connection_t
*conn) {
    pthread_mutex_lock(&manager.recursos[tipo].mutex);

    if (manager.recursos[tipo].available_amount >= amount) {
        // Hay recursos disponibles → conceder inmediatamente
        manager.recursos[tipo].available_amount -= amount;
        result = RM_GRANTED;
    } else {
        // No hay suficientes recursos → encolar solicitud FIFO
        queue_enqueue(&manager.recursos[tipo].cola, job_id, amount, conn);
        result = RM_QUEUED;
    }

    pthread_mutex_unlock(&manager.recursos[tipo].mutex);
    return result;
}
```

- **Mecanismo de Liberación y Atención de Cola:** la función release_resource() es la encargada de que cuando se recibe un RELEASE (desde otro nodo) o un JOB_RELEASE (desde Erlang), el sistema no solo libera los recursos, sino que también atiende tantas solicitudes encoladas como sea posible con los recursos liberados en orden FIFO insertando los Jobs en la tabla de Jobs mediante tabla_jobs_insert() para cada solicitud atendida, evitando “Jobs fantasma” y notifica a los clientes enviando GRANTED o

JOB_GRANTED según corresponda. Además, `release_resource()` libera el lock del recurso antes de interactuar con la tabla de Jobs para prevenir interbloques.

- **Manejo de Desconexiones y Limpieza:** Ante una desconexión inesperada (por `E POLLERR`, `E POLLHUP` o `read() == 0`), el sistema ejecuta una limpieza exhaustiva a través de la función `process_disconnect()` la cual realiza:
 1. Elimina solicitudes pendientes en las colas FIFO de los tres tipos de recursos mediante `queue_delete_by_conn()`.
 2. Libera todos los recursos asignados a los jobs de esta conexión utilizando `tabla_jobs_delete_by_conn()`, que a su vez invoca `release_resource()` para cada asignación
 3. Limpia el rastreador de dueños de jobs (`job_owners`) para eliminar referencias a la conexión desconectada.
 4. Elimina solicitudes salientes pendientes de la lista `pendientes_salientes`
 5. Elimina la conexión de la tabla de conexiones activas mediante `tabla_conns_delete_by_conn(conn)`
- **Mantenimiento de la Tabla de Nodos:** la tabla se actualiza mediante dos mecanismos complementarios:
 1. Recepción de Anuncios: utilizando la función `process_announce()` se procesa el mensaje de la facha "ANNOUNCE <puerto> cpu:X mem:Y gpu:Z" y se inserta o actualiza en la tabla con `tabla_nodos_insert_or_update()`
 2. Purga periódica: utilizando la función `tabla_nodos_purge()` recorre los nodos para eliminar los nodos cuyo `last_seen` supere el `timeout_secs`. Esta purga se ejecuta cada 5 segundo desde el hilo de broadcast (`broadcast_loop`), asegurando que la tabla siempre refleje el estado actual de la red
- **Tabla de Conexiones:** para optimizar las comunicaciones, el sistema mantiene una tabla que mapea direcciones IP a conexiones TCP activas:

```
typedef struct {  
    ConnEntry *buckets[TAM_TABLA_CONN]; // Hash table  
    pthread_mutex_t mutex;  
} TablaConns;
```

De esta manera, se mantiene un registro de las conexiones activas para poder hacer el envío de mensajes acorde
- **Rastreo de Dueños de Jobs:** el sistema mantiene un registro que asocia cada `job_id` con la conexión original del cliente que lo solicito:

```
typedef struct JobOwner {  
    int job_id; //ID del job  
    connection_t *conn; // Conexión original (Erlang que hizo JOB_REQUEST)  
    struct JobOwner *next;  
} JobOwner;
```

De esta manera, cuando llega un GRANTED o DENIED desde un nodo remoto, el sistema sabe a qué cliente Erlang reenviarlo mediante `buscar_job_owner()`. Esto permite notificar al planificador original sobre el resultado de su solicitud, incluso si la conexión hacia el nodo remoto se estableció de forma asíncrona. La estructura se limpia automáticamente cuando el job finaliza o la conexión se cierra (`eliminar_job_owner()`)
- **Mecanismo de Solicitudes Remota Pendientes:**

```
typedef struct OutReq {
```

```

connection_t *conn; // Conexión hacia el nodo remoto
char msg[BUFF_SIZE]; // Mensaje RESERVE pendiente
char ip[16]; // IP del nodo destino
struct OutReq *next;
} OutReq;

```

Cuando un JOB_REQUEST requiere recursos en un nodo remoto y no hay una conexión activa, el sistema inicia `connect_remote_node()` de forma asíncrona y encola el mensaje en `pendientes_salientes`. Una vez que `process_connection_ready()` confirma la conexión exitosa, se envía el mensaje encolado y se agrega la conexión a `tabla_conns` para futuras reutilizaciones. Si la conexión falla (`process_connection_failed()`), los mensajes encolados se descartan y se notifica al planificador con `JOB_DENIED`.

- **Mecanismo de Reserva Secuencial** : El gestor de recursos implementa un mecanismo de reserva secuencial donde las solicitudes de un JOB_REQUEST se procesan en cadena, una tras otra. Esto garantiza que un job solo se considere "concedido" cuando todos sus recursos han sido reservados exitosamente. Ver ejemplo:

```

1. Erlang envía JOB_REQUEST 1001 @A:cpu:2 @B:gpu:1

2. Agente C: registrar_job_owner(1001, conn_erlang,
[cpu:2, gpu:1])

3. Agente C: avanzar_reserva(1001)

4. avanzar_reserva(1001):
  a. Extrae primer request: cpu:2 (local)
  b. reserve_resource(cpu:2) → GRANTED
  c. tabla_jobs_insert(1001, cpu:2)
  d. avanzar_reserva(1001) ← Recursivo

```

```

5. avanzar_reserva(1001):
  a. Extrae siguiente request: gpu:1 (remoto)
  b. Envía RESERVE 1001 gpu 1 a B
  c. Espera GRANTED de B

6. B → A: GRANTED 1001

7. Agente C: process_message("GRANTED 1001")

8. Agente C: avanzar_reserva(1001)

9. avanzar_reserva(1001):
  a. No hay más requests pendientes
  b. Envía JOB_GRANTED 1001 a Erlang
  c. eliminar_job_owner(1001)

```

El planificador Erlang, implementado en `c_agent_client.erl` es el encargado de generar Jobs, solicitar recursos a los nodos del cluster y coordinar la ejecución de tareas. Su diseño se basa en un modelo de actores concurrentes que se comunican mediante paso de mensajes, aprovechando las capacidades nativas de concurrencia de Erlang. A continuación, hablaremos de los componentes principales del planificador:

- **Master Process (Proceso coordinador)**: el master es el proceso central que orquesta todo el sistema. Se registra globalmente con el nombre master para que otros procesos puedan comunicarse con él. Las responsabilidades del master son
 1. Mantener la lista de nodos disponibles (`Maps_ist`)

2. Mantener el mapa de Jobs activos (HandlersMap)
 3. Generar ráfagas de Jobs cuando el mapa esta vacío
 4. procesar respuestas del agente C
 5. gestionar liberaciones de Jobs
 6. registrar eventos en el archivo de log
- **Job Handlers (Procesos de Jobs):** cada job es manejado por un proceso independiente que sigue un ciclo de vida simple (empleada por `job_handler()`). Espera un mensaje, si es `Granted` el job simula trabajo y luego notifica al master, si es `denied` el job termina inmediatamente y notifica al master para limpieza y si es `timeout`, si no se recibe respuesta en 5 segundos, el job ahorta y notifica al master
 - **Node Mappers (Procesos de Parsing):** procesan la información de nodos recibida del agente C y la convierten en estructuras de datos manejables. Convierte una lista de strings en un mapa
 - **Generación de Jobs y Selección de Recursos:** el planificador genera Jobs (con `aramar_comando_ordenad()`) con solicitudes de recursos aleatorias pero balanceadas, pues poseen aleatoriedad, los nodos se recorren en orden ascendente de IP, ignora nodos vacíos y se realizan ráfagas controladas de generación de Jobs que se lanzan simultáneamente a través de la función `disparar_rafaga()`.
 - **Comunicación con el agente C:** el planificador se comunica con el agente C mediante mensajes de texto ASCII sobre TCP
 - **Registro de eventos en Log:** el planificador mantiene un archivo de log con todos los eventos relevantes (función `registrar_log()`)

Ejecución del Planificador Erlang

Al iniciar el planificador, se intenta conectar al agente C en el puerto 8000 de localhost. Se hace con cadenas en formato 'list', permitiendo pattern matching de la forma `Cadena1 ++ Cadena2 ++ ... + CadenaN`, datos entrantes sin formato y conexión en modo pasivo. Se registra el proceso principal como 'master'.

Luego, se envía la cadena `"GET_NODES\n"` al agente C para solicitar la información sobre nodos disponibles. Si se recibe una cadena que inicie con `"NODES "`, el programa asume que la información se recibió correctamente y continúa su ejecución, creando la lista de mapas donde cada uno representa uno de los nodos recibidos.

A continuación, se crea otro mapa (inicialmente vacío) `HandlersMap` que almacenará los jobs pendientes de respuesta y el proceso encargado de cada uno. Una vez hecho esto, 'master' entra en el loop donde se dedica constantemente a:

- Enviar una nueva ráfaga de jobs al agente C, que permanecerán en el mapa de jobs hasta que se reciba una respuesta, y crear un proceso por cada job
- Recibir un mensaje de C que puede ser `JOB_GRANTED`, `JOB_DENIED` o `JOB_TIMEOUT` junto con el `job_id` correspondiente
- Notificar al proceso correspondiente sobre la asignación, denegación o expiración de su solicitud, además de registrar dicha notificación en el log utilizando `registrar_log()`

- Eliminar dicho job del mapa de jobs
- Repetir el bucle a menos que ocurra un error de conexión con el agente C distinto a 'timeout'

En paralelo, los procesos correspondientes a cada job simulan trabajar si se les otorga los recursos, suspendiendo la ejecución unos segundos antes de solicitar la liberación de los mismos y terminar.

Estrategia contra deadlock

En el caso mencionado en el enunciado del trabajo práctico, identificamos que el problema está relacionado con el orden de solicitud de los recursos, ya que se solicitan los mismos en distinto orden.

Para evitar un deadlock en este caso, decidimos garantizar un orden de recepción de solicitudes basado en el valor de las direcciones IP de cada nodo. Esta estrategia asegura que todos los Jobs soliciten recursos en el mismo orden, eliminando la posibilidad de ciclos de espera. La estrategia se basa en el principio de prevención de deadlock por ordenación de recursos. Si cada nodo tiene un orden global basado en su IP los Jobs siempre solicitan recursos en orden ascendente de IP entonces como todos siguen el mismo orden, nunca se forma un ciclo de espera. Podemos verlo en la siguiente demostración:

Nodos: A (192.168.1.10), B (192.168.1.20), C (192.168.1.30)

Orden global: $A < B < C$

Regla: Todo job solicita recursos en el orden $A \rightarrow B \rightarrow C$

Análisis de deadlock:

- Para que haya deadlock, debe existir un ciclo de espera
 - Un ciclo requiere que algún job solicite recursos en orden descendente
 - Como todos usan el orden $A \rightarrow B \rightarrow C$, no hay ciclos
- Deadlock imposible**

Ejemplo paso a paso que demuestra como se evita un deadlock con dos nodos (situación de deadlock planteada en el enunciado)

Nodos: A (192.168.1.10), B (192.168.1.20)

Orden global: A → B

Job1: solicita recursos en orden A → B

Job2: solicita recursos en orden A → B

→ No hay posibilidad de que Job1 espere a B mientras Job2 espera a A

→ Deadlock imposible

Diagrama de secuencia de la comunicación (local y remoto)

